

MODUL 13

(GRAPH I)

DESKRIPSI TEMA

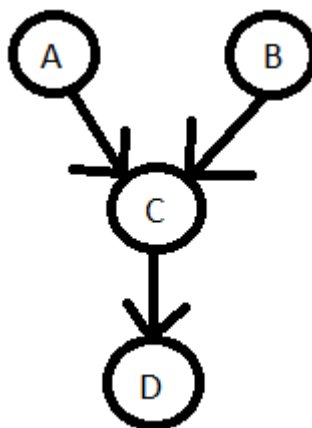
Depth First Search (DFS) adalah algoritma untuk menjelajahi ataupun mencari pada suatu tree atau graph. Cara kerja algoritma ini adalah dengan menjelajahi sejauh mungkin setiap cabang pada suatu *node*, lalu jika sudah tidak bisa ditelusuri lagi akan mundur ke node sebelumnya.

Pada praktikum minggu ini, DFS akan diimplementasikan ke dalam graph. Yang membedakan antara graph dan tree adalah adanya *cycles* di dalam graph, sehingga dimungkinkan untuk kembali ke *node* yang sama lagi. Oleh karena itu, untuk menghindari pemrosesan *node* lebih dari satu kali, kita menggunakan variabel *visited* untuk menandai bahwa sebuah *node* sudah dikunjungi atau belum.

Ada 2 metode DFS yang dapat diimplementasikan, yaitu dengan menggunakan struktur data **Stack** dan juga dapat menggunakan fungsi **Recursive**. Pada praktikum minggu ini, DFS akan diimplementasikan menggunakan fungsi *recursive*.

Breadth First Search (BFS) adalah algoritma untuk menelusuri atau mencari struktur data pohon atau grafik. Berbeda dengan DFS, cara kerja algoritma BFS ini dengan mengeksplorasi/menyebar ke semua node tetangga di suatu level/kedalaman sebelum pindah ke node di tingkat kedalaman berikutnya. Metode yang dapat mengimplementasikan BFS adalah dengan menggunakan struktur data *Queue*.

Topological Sort adalah algoritma sortir yang hanya bisa digunakan jika graph tidak mempunyai *directed cycles*, atau yang disebut dengan Directed Acyclic Graph (DAG). Topological Sort mengimplementasikan DFS dan Stack. Pada DFS dilakukan print node, lalu memanggil fungsi secara rekursif. Namun, pada topological sort dilakukan pemanggilan fungsi secara rekursif, hingga tidak dapat ditelusuri lagi lalu dimasukkan ke dalam stack.



Graph diatas, jika diurutkan menggunakan topological sort mempunyai 2 hasil yang valid, yaitu:

- B, A, C, D

- A, B, C, D

Oleh karena itu, yang terpenting dari Hasil akhir sortir ini adalah membuat setiap edge uv , yaitu dari node v ke node u , u akan muncul duluan sebelum v di dalam pengurutan.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa dapat memahami dan menerapkan konsep graph dan algoritma shortest path (C_3).

PENUNJANG PRAKTIKUM

1. Software CodeBlocks

LANGKAH-LANGKAH PRAKTIKUM

Implementasi DFS

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Edge {
5      int dest;
6      struct Edge *next;
7  };
8
9
10 void addEdge(struct Edge *adjList[], int src, int dest) {
11     struct Edge *tmp;
12
13     tmp = (struct Edge *) malloc(sizeof(struct Edge));
14     tmp->dest = dest;
15     tmp->next = NULL;
16
17     if(adjList[src] == NULL) {
18         adjList[src] = tmp;
19     }
20
21     else {
22         struct Edge *ptr = adjList[src];
23         while(ptr->next != NULL) {
24             ptr = ptr->next;
25         }
26
27         ptr->next = tmp;
28     }
29 }
30
```

```
31 void DFS(int v, int visited[], struct Edge *adjList[]) {
32     int i, j;
33
34     visited[v] = 1;
35     printf("%d ", v);
36
37     struct Edge *it;
38     for(it = adjList[v]; it != NULL; it = it->next) {
39         if(!visited[it->dest]) {
40             DFS(it->dest, visited, adjList);
41         }
42     }
43 }
44
```

```

45 int main(int argc, const char* argv[]) {
46     int i, j, k;
47
48     int V; //Jumlah Vertex dalam Graph
49     struct Edge *adjList[100]; //Representasi Graph dengan Adjacency List
50
51
52     //Initialize Graph
53     printf("Banyak Vertex: ");
54     scanf("%d", &V);
55     printf("\n");
56
57     for(int i = 0; i < V+2; i++) {
58         adjList[i] = NULL;
59     }
60
61
62     //Insert Edge
63     int src;
64     int dest;
65     i = 0;
66     while(1) {
67         printf("Adjacency List ke-%d\n", ++i);
68         printf("Source: ");
69         scanf("%d", &src);
70
71         printf("Destination: ");
72         scanf("%d", &dest);
73
74         printf("\n");
75
76         if(src <= -1 || dest <= -1) {
77             break;
78         }
79
80         else {
81             addEdge(adjList, src, dest);
82             addEdge(adjList, dest, src);
83         }
84     }
85

```

```

87     //Initialize visited node
88     int visited[V+2];
89     for(i = 0; i < V+2; i++) {
90         visited[i] = 0;
91     }
92
93
94     //Call DFS
95     int start;
96     printf("Starting Node: ");
97     scanf("%d", &start);
98     DFS(start, visited, adjList);
99 }

```

Tutorial DFS

1. Jika function main() bagian Insert Edge pada program sebelumnya dimodifikasi dan diberi input sebagai berikut:

4 0 1 0 2 1 2 2 0 2 3 3 3 -1 -1 3

Apakah output yang diberikan function DFS() dari program ini?

```

62      //Insert Edge
63      int src;
64      int dest;
65      i = 0;
66      while(1) {
67          printf("Adjacency List ke-%d\n", ++i);
68          printf("Source: ");
69          scanf("%d", &src);
70
71          printf("Destination: ");
72          scanf("%d", &dest);
73
74          printf("\n");
75
76          if(src <= -1 || dest <= -1) {
77              break;
78          }
79
80          else {
81              addEdge(adjList, src, dest);
82              //addEdge(adjList, dest, src);
83          }
84      }

```

2. Apakah ada perbedaan output dengan program sebelumnya? Sertakan alasan anda!
3. Simpan pada file dengan nama **Week[xx]_[nim]_[nama]_o1.txt**
e.g.: **Week13_10101_LionelReinhartHalim_o1.txt**

Implementasi BFS

Note:

- Gunakan fungsi struktur data queue yang telah anda pelajari pada praktikum sebelumnya.
- Gunakan fungsi addEdge() pada tutorial DFS sebelumnya.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Edge {
5      int dest;
6      struct Edge *next;
7  };
8
9  struct Queue {
10     int data;
11     struct Queue *next;
12 };
13
14 // Function Prototype-----
15 void addEdge(struct Edge *adjList[], int src, int dest);
16 void BFS(int v, int visited[], struct Edge *adjList[]);
17 void enqueue(struct Queue **queue, int v);
18 void dequeue(struct Queue **queue);
19 int isEmpty(struct Queue *queue);
20 int front(struct Queue *queue);
21 // END Function Prototype-----
22

```

```

23 void BFS(int start, int visited[], struct Edge *adjList[])
24 {
25     struct Queue *queue = NULL;
26
27     // tandai vertex start nya sudah dikunjungi, lalu masukkan ke queue
28     visited[start] = 1;
29     enqueue(&queue, start);
30
31     while(!isEmpty(queue))
32     {
33         // print lalu buang vertex dari queue
34         int v = front(queue);
35         printf("%d ", v);
36         dequeue(&queue);
37
38         // dapatkan semua adjacent vertex yang telah di dequeue
39         // kalau belum di visit, maka tandai visited nya dan enqueue
40         struct Edge *it;
41         for(it = adjList[v]; it != NULL; it = it->next){
42             if(!visited[it->dest]){
43                 visited[it->dest] = 1;
44                 enqueue(&queue, it->dest);
45             }
46         }
47     }
48 }
49

```

```
50 int main(int argc, const char* argv){
51     int i;
52     int V; // Jumlah Vertex dalam Graph
53     struct Edge *adjList[100]; // Representasi Graph dengan Adjacency List
54
55     // Initialize Graph
56     printf("Banyak Vertex: ");
57     scanf("%d", &V);
58     printf("\n");
59
60     for(i = 0; i < V+2; i++){
61         adjList[i] = NULL;
62     }
63
64     // Insert Edge
65     int src, dest;
66     i = 0;
67     while(1){
68         printf("Adjacency List ke-%d\n", ++i);
69         printf("Source: ");
70         scanf("%d", &src);
71
72         printf("Destination: ");
73         scanf("%d", &dest);
74
75         printf("\n");
76
77         if(src <= -1 || dest <= -1){
78             break;
79         }
80         else{
81             addEdge(adjList, src, dest);
82             addEdge(adjList, dest, src);
83         }
84     }
85
86     // Initialize visited node
87     int visited[V+2];
88     for(i = 0; i < V+2; i++){
89         visited[i] = 0;
90     }
91
92     // Call BFS
93     int start;
94     printf("Starting Node: ");
95     scanf("%d", &start);
96     BFS(start, visited, adjList);
97
98     return 0;
99 }
100
```

Tutorial BFS

- 1) Jika function main() bagian Insert Edge pada program sebelumnya dimodifikasi dan diberi input sebagai berikut:

4 0 1 0 2 1 2 2 0 2 3 3 3 -1 -1 3

```

62 //Insert Edge
63 int src;
64 int dest;
65 i = 0;
66 while(1) {
67     printf("Adjacency List ke-%d\n", ++i);
68     printf("Source: ");
69     scanf("%d", &src);
70
71     printf("Destination: ");
72     scanf("%d", &dest);
73
74     printf("\n");
75
76     if(src <= -1 || dest <= -1) {
77         break;
78     }
79
80     else {
81         addEdge(adjList, src, dest);
82         //addEdge(adjList, dest, src);
83     }
84 }

```

Apakah output yang diberikan function BFS() dari program ini?

- 2) Apakah ada perbedaan output dengan program sebelumnya? Sertakan alasan anda!
- 3) Simpan pada file dengan nama **Week[xx]_[nim]_[nama]_01.txt**
e.g.: **Week13_10101_BudimanPutraJaya_02.txt**

Implementasi Topological Sort

Note: Gunakan fungsi stack yang telah anda pelajari pada praktikum sebelumnya

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Edge {
5      int dest;
6      struct Edge *next;
7  };
8
9  struct Stack {
10     int data;
11     struct Stack *next;
12 };
13
14
15 //Function Prototype-----
16 void addEdge(struct Edge *adjList[], int src, int dest);
17 void topologicalSort(int v, int visited[], struct Edge *adjList[], struct Stack **stack);
18 void push(struct Stack **stack, int v);
19 int isEmpty(struct Stack *stack);
20 int top(struct Stack *stack);
21 void pop(struct Stack **stack);
22 //END OF Function Prototype-----
```

```

26 void addEdge(struct Edge *adjList[], int src, int dest) {
27     struct Edge *tmp;
28
29     tmp = (struct Edge *) malloc(sizeof(struct Edge));
30     tmp->dest = dest;
31     tmp->next = NULL;
32
33     if(adjList[src] == NULL) {
34         adjList[src] = tmp;
35     }
36
37     else {
38         struct Edge *ptr = adjList[src];
39         while(ptr->next != NULL) {
40             ptr = ptr->next;
41         }
42
43         ptr->next = tmp;
44     }
45 }
46
47 void topologicalSort(int v, int visited[], struct Edge *adjList[], struct Stack **stack) {
48     visited[v] = 1;
49
50     struct Edge *it;
51     for(it = adjList[v]; it != NULL; it = it->next) {
52         if(!visited[it->dest]) {
53             topologicalSort(it->dest, visited, adjList, stack);
54         }
55     }
56
57     push(stack, v);
58 }

```

```

101 int main(int argc, const char *argv[]) {
102     int i;
103     int V; //Jumlah Vertex dalam Graph
104     struct Edge *adjList[100]; //Representasi Graph dengan Adjacency List
105     struct Stack *stack = NULL;
106
107     // Initialize Graph
108     printf("Banyak Vertex: ");
109     scanf("%d", &V);
110     printf("\n");
111
112     //Initialize Adjacency List
113     for(int i = 0; i < 100; i++) {
114         adjList[i] = NULL;
115     }
116

```

```

117 //Insert Edge
118 int src, dest;
119 i = 0;
120 while(1) {
121     printf("Adjacency List ke-%d\n", ++i);
122     printf("Source: ");
123     scanf("%d", &src);
124
125     printf("Destination: ");
126     scanf("%d", &dest);
127
128     printf("\n");
129
130     if(src <= -1 || dest <= -1) {
131         break;
132     }
133     else{
134         addEdge(adjList, src, dest);
135     }
136 }
137

```

```

138 //Initialize visited node
139 int visited[V+2];
140 for(i = 0; i < V+2; i++) {
141     visited[i] = 0;
142 }
143
144 //Call Topological Sort
145 for(i = 0; i < V; i++) {
146     if(!visited[i]) {
147         topologicalSort(i, visited, adjList, &stack);
148     }
149 }
150
151 //Print the result
152 while(!isEmpty(stack)) {
153     printf("%d ", top(stack));
154     pop(&stack);
155 }
156
157 return 0;
158 }

```

Tutorial Topological Sort

1. Jika program diberi input sebagai berikut:

6 5 2 5 0 4 0 4 1 2 3 3 1 -1 -1

Apakah output dari input tersebut? Jelaskan apa yang terjadi pada fungsi topologicalSort!

2. Simpan pada file dengan nama **Week[xx]_[nim]_[nama]_03.txt**

e.g.: **Week13_10101_LionelReinhartHalim_03.txt**

Challenge 1 – BFS & DFS

Selain menggunakan Adjacency List, Graph dapat di representasikan dengan menggunakan Adjacency Matrix.

Ubahlah *code* pada tutorial BFS dan DFS dengan menggunakan Adjacency Matrix!

Kumpulkan dengan nama file:

Week[xx]_[nim]_[nama]_challengeBFS.c

e.g.: **Week13_10101_BudimanPutraJaya_challengeBFS.c**

Week[xx]_[nim]_[nama]_challengeDFS.c

e.g.: **Week13_10101_BudimanPutraJaya_challengeDFS.c**

Challenge 2 – Topological Sort

Buatlah sebuah program yang menerima input Banyak Materi, Nama setiap Materinya, dan hubungan dari satu materi dengan materi lainnya sampai menerima input “done”, lalu tampilkan hasil dari algoritma topological sort.

Kumpulkan dengan nama file:

Week[xx]_[nim]_[nama]_challengeTopoSort.c

e.g.: **Week13_10101_BudimanPutraJaya_challengeTopoSort.c**

Contoh Program:

```
Banyak Materi: 14
Nama materi ke-1: Pointer
Nama materi ke-2: Array
Nama materi ke-3: Struct
Nama materi ke-4: Linked List
Nama materi ke-5: Radix Sort
Nama materi ke-6: Hashing
Nama materi ke-7: Stack
Nama materi ke-8: Queue
Nama materi ke-9: Tree
Nama materi ke-10: UFDS
Nama materi ke-11: Graph
Nama materi ke-12: BFS
Nama materi ke-13: DFS
Nama materi ke-14: Topological Sort
```

Hubungan materi Algoritma & Struktur Data ke-1

Materi yang harus dipelajari dulu: Pointer

Materi yang dapat dipelajari setelahnya: Array

Hubungan materi Algoritma & Struktur Data ke-2

Materi yang harus dipelajari dulu: Array

Materi yang dapat dipelajari setelahnya: Linked List

Hubungan materi Algoritma & Struktur Data ke-3

Materi yang harus dipelajari dulu: Struct

Materi yang dapat dipelajari setelahnya: Linked List

Hubungan materi Algoritma & Struktur Data ke-4

Materi yang harus dipelajari dulu: Linked List

Materi yang dapat dipelajari setelahnya: Radix Sort

Hubungan materi Algoritma & Struktur Data ke-5

Materi yang harus dipelajari dulu: Linked List

Materi yang dapat dipelajari setelahnya: Hashing

Hubungan materi Algoritma & Struktur Data ke-6

Materi yang harus dipelajari dulu: Linked List

Materi yang dapat dipelajari setelahnya: Stack

Hubungan materi Algoritma & Struktur Data ke-7

Materi yang harus dipelajari dulu: Linked List

Materi yang dapat dipelajari setelahnya: Queue

Hubungan materi Algoritma & Struktur Data ke-8

Materi yang harus dipelajari dulu: Linked List

Materi yang dapat dipelajari setelahnya: Tree

Hubungan materi Algoritma & Struktur Data ke-9

Materi yang harus dipelajari dulu: Tree

Materi yang dapat dipelajari setelahnya: UFDS

Hubungan materi Algoritma & Struktur Data ke-10

Materi yang harus dipelajari dulu: Queue

Materi yang dapat dipelajari setelahnya: BFS

Hubungan materi Algoritma & Struktur Data ke-11

Materi yang harus dipelajari dulu: Stack

Materi yang dapat dipelajari setelahnya: DFS

Hubungan materi Algoritma & Struktur Data ke-12

Materi yang harus dipelajari dulu: Tree

Materi yang dapat dipelajari setelahnya: Graph

Hubungan materi Algoritma & Struktur Data ke-13

Materi yang harus dipelajari dulu: Stack

Materi yang dapat dipelajari setelahnya: Topological Sort

Hubungan materi Algoritma & Struktur Data ke-14

Materi yang harus dipelajari dulu: Graph

Materi yang dapat dipelajari setelahnya: BFS

Hubungan materi Algoritma & Struktur Data ke-15

Materi yang harus dipelajari dulu: Graph

Materi yang dapat dipelajari setelahnya: DFS

Hubungan materi Algoritma & Struktur Data ke-16

Materi yang harus dipelajari dulu: Graph

Materi yang dapat dipelajari setelahnya: Topological Sort

Hubungan materi Algoritma & Struktur Data ke-17

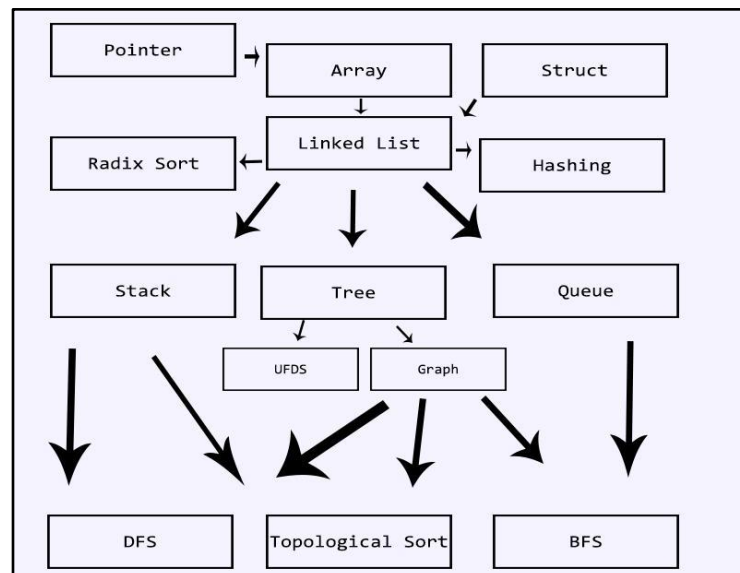
Materi yang harus dipelajari dulu: done

Materi yang dapat dipelajari setelahnya: done

Contoh hasil output program:

```
Urutan materi yang harus dipelajari:
1. Struct
2. Pointer
3. Array
4. Linked List
5. Tree
6. Graph
7. UFDS
8. Queue
9. BFS
10. Stack
11. Topological Sort
12. DFS
13. Hashing
14. Radix Sort
```

Contoh visualisasi inputan program diatas:



REFERENSI